PROPUESTA DE BACKEND

Sistema de reservas para barbería

Documento de definición funcional y arquitectura de referencia para revisión del equipo backend.

Objetivo: establecer cómo debería comportarse el backend, qué responsabilidades debe cubrir y qué reglas debe garantizar, sin imponer todavía un lenguaje, framework o proveedor tecnológico.

Alcance	Definición
Negocio	Una barbería, atendida inicialmente por un único barbero (el propietario).
Canales	Panel web privado para el dueño y aplicación móvil para clientes.
Backend	Una API compartida por ambos canales; una única fuente de verdad.
Persistencia	Base de datos relacional propuesta: PostgreSQL (a validar con el equipo).
Estado del documento	Propuesta para discusión; no es una especificación cerrada ni código de implementación.
Fecha	9 de noviembre de 2026

Principio de diseño

Resolver primero reservas, disponibilidad, pagos, cancelaciones y seguridad. Evitar desde la primera versión conceptos de SaaS, múltiples sucursales, múltiples barberos, inventario o comisiones internas que no forman parte del alcance actual.

1. Resumen ejecutivo

Se propone construir un backend central que exponga capacidades de autenticación, catálogo, disponibilidad, reservas, pagos, reembolsos y administración. La app móvil y el panel web consumen la misma API; las reglas críticas viven en el backend y no se duplican en los clientes.

Componente	Responsabilidad principal
App móvil de clientes	Registro, verificación, inicio de sesión, catálogo, consulta de disponibilidad, reserva, pago, historial, cancelación y perfil.
Panel web del dueño	Administrar perfil del negocio, servicios, horarios, excepciones, calendario, citas, clientes y sesiones.
API / backend	Autorización, validación de reglas, cálculo de disponibilidad, creación segura de reservas, integración con proveedor de pagos y procesos programados.
Base de datos	Persistir usuarios, configuración, servicios, horarios, excepciones, citas, pagos, reembolsos y sesiones.
Proveedor de pagos	Procesar pagos y reembolsos; notificar cambios mediante webhooks firmados.
Servicio de correo	Enviar verificación de correo y códigos de recuperación de contraseña.

1.1 Principios

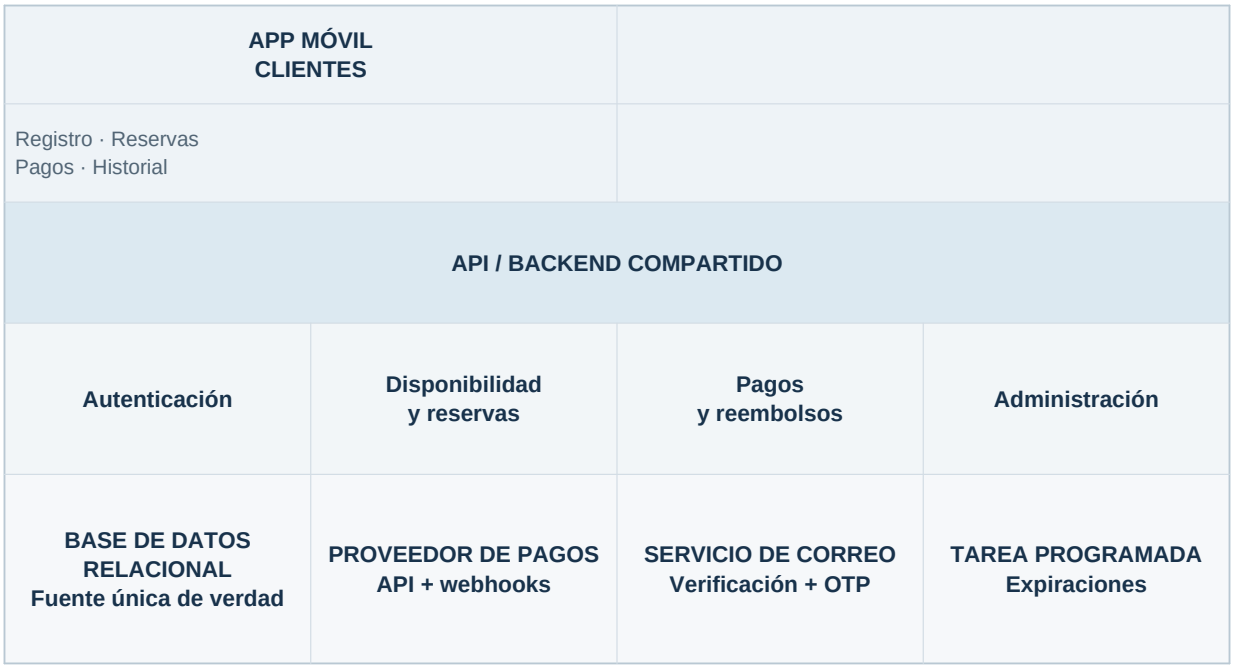
- Una sola barbería y un solo calendario operativo en la primera versión.
- La API es la autoridad: la app no decide si un horario está libre ni confirma un pago por sí misma.
- Las operaciones de reserva y pago deben ser seguras ante solicitudes simultáneas, reintentos y webhooks duplicados.
- Los importes, duración y datos relevantes del servicio se conservan como fotografía histórica en cada cita.
- Las fechas se almacenan como instantes con zona horaria; la zona del negocio controla la presentación y la interpretación de horarios.
- Mantener la solución preparada para evolucionar, pero sin implementar ahora multi-sucursal ni multi-empleado.

1.2 Fuera del alcance inicial

- SaaS multi-tenant, sucursales o múltiples barberos.
- Inventario, comisiones, nómina, cupones, reseñas y programas de lealtad.
- Carrito o reserva de varios servicios en una misma cita; se propone un servicio por cita en V1.
- Reserva pública desde el sitio web informativo: el canal de reserva del cliente será la app móvil. El panel web es privado para el dueño.

2. Arquitectura lógica propuesta

La arquitectura puede implementarse como un monolito modular al inicio. No hay una necesidad funcional actual que justifique microservicios. Los módulos pueden compartir despliegue y base de datos, manteniendo límites claros en el código.



2.1 Módulos de backend

Módulo	Responsabilidades
Identidad y acceso	Registro, login, verificación, recuperación, refresh de sesión, autorización por rol y revocación.
Negocio y servicios	Perfil único, zona horaria, moneda, catálogo, precios, duraciones y activación de servicios.
Agenda	Horarios semanales, intervalos múltiples por día, excepciones, cierres y bloqueos.
Disponibilidad	Generar inicios posibles y excluir citas, buffers, bloqueos y reservas temporales vigentes.
Reservas	Crear, confirmar, cancelar, completar, marcar no-show y conservar historial.
Pagos	Crear intención/sesión de pago, validar notificaciones, conciliar estados y aplicar idempotencia.
Reembolsos	Solicitar reembolso completo, registrar estado del proveedor y mostrar resultado.
Administración	Operaciones del dueño, consulta de clientes/sesiones y acciones auditables.

3. Reglas de negocio que debe garantizar el backend

Estas reglas son el núcleo del sistema. Deben centralizarse en el backend para que se apliquen igual desde la app móvil y desde el panel.

3.1 Horarios y disponibilidad

- El dueño define libremente sus horarios habituales por día de la semana.
- Cada día admite varios intervalos independientes; por ejemplo, 09:00–13:00 y 14:00–19:00.
- El dueño puede definir días cerrados, horarios especiales y bloqueos por fecha/hora.
- Un bloqueo nuevo no debe guardarse silenciosamente si se cruza con citas existentes: el panel debe advertirlo y exigir que el dueño resuelva esas citas.
- La disponibilidad se calcula en tiempo real a partir del horario, excepciones, servicios, citas activas y tiempo de preparación.
- No es necesario almacenar una tabla permanente de todos los slots: pueden generarse bajo demanda.

3.2 Duración y preparación

Cada servicio tiene una duración. Además, se aplica un tiempo de preparación/limpieza posterior a cada cita; el valor inicial acordado es de 10 minutos. Ejemplo: servicio de 16:00 a 16:30; el siguiente inicio permitido será 16:40.

- El buffer ocupa agenda y no se ofrece como horario disponible.
- La cita conserva la duración y el buffer aplicados cuando fue creada (snapshot).
- Si el dueño cambia el buffer general en el futuro, las citas existentes no se modifican.
- El intervalo de inicio de slots (inicialmente 10 minutos) es distinto de la duración del servicio y del buffer; el dueño/equipo deberá confirmar si será configurable por el dueño o solo por configuración del negocio.

3.3 Reserva temporal y pago

- Al iniciar una reserva, el backend crea una retención temporal con estado PENDING_PAYMENT.
- La retención dura 10 minutos desde su creación. Durante ese tiempo, el horario no puede asignarse a otra cita.
- Si el pago no se confirma a tiempo, la reserva pasa a EXPIRED y libera el horario.
- Solo el backend, tras validar una notificación auténtica del proveedor o una consulta confiable, cambia la cita a CONFIRMED.
- Los webhooks pueden llegar repetidos, retrasados o fuera de orden: procesarlos de forma idempotente y comprobar el estado actual antes de confirmar.
- Un webhook tardío no debe confirmar una reserva ya expirada ni desplazar una cita que haya tomado ese horario.

3.4 Evitar doble reserva

La comprobación de disponibilidad seguida de la inserción debe ser atómica. Dos clientes podrían seleccionar el mismo horario al mismo tiempo; la base de datos y la transacción deben impedir que ambos lo obtengan.

- Usar transacciones y un mecanismo de serialización/lock apropiado para el calendario único.
- Añadir una protección de integridad contra solapamientos de citas activas en la base de datos cuando la tecnología elegida lo permita.
- Al crear un bloqueo, validar dentro de la misma operación que no existan citas incompatibles.
- Al crear una cita, volver a validar horarios y bloqueos; nunca confiar en la disponibilidad que vio el cliente minutos antes.

4. Cancelaciones, reembolsos y tiempo

4.1 Cancelación por parte del cliente

- Regla acordada: el cliente puede cancelar y solicitar reembolso completo hasta 24 horas antes del inicio de la cita.
- La regla se calcula como 24 horas exactas antes del instante de inicio, no como “el día calendario anterior”.
- Dentro de las 24 horas previas, el cliente no puede cancelar desde la app ni solicitar reembolso automático.
- El backend debe validar esta condición; ocultar el botón en la app no es suficiente.

4.2 Cancelación por parte del dueño

- El dueño puede cancelar una cita desde el panel.
- Al cancelar una cita pagada/confirmada, el sistema debe iniciar automáticamente un reembolso del 100 % al cliente.
- La comisión que el procesador no devuelva será asumida por el negocio, según la política acordada.
- El estado de la cita y el estado del reembolso son conceptos separados: una cita puede estar CANCELLED mientras su reembolso sigue PENDING.
- La cancelación debe liberar el horario una vez que la cita quede cancelada; el proceso de reembolso puede continuar de forma asíncrona.

4.3 Modelo de estados sugerido

Entidad	Estados principales
Cita	PENDING_PAYMENT · CONFIRMED · COMPLETED · CANCELLED · EXPIRED · NO_SHOW
Pago	PENDING · AUTHORIZED · COMPLETED · FAILED · CANCELLED · PARTIALLY_REFUNDED · REFUNDED
Reembolso	PENDING · COMPLETED · FAILED

No mezclar el estado de pago con el estado de la cita. Una cita cancelada no debe volver a confirmarse por una notificación de pago tardía. Si el proveedor reporta un cobro después de expirar la retención, debe existir una política de conciliación: intentar reembolsar automáticamente y registrar el incidente, o escalarlo al dueño.

4.4 Zona horaria

- Mantener el servidor/VPS en UTC; no cambiar su reloj para ajustarlo al negocio.
- Guardar los instantes de citas, pagos, expiraciones y sesiones con tipo equivalente a TIMESTAMPTZ.
- Guardar una zona IANA configurable para el negocio; valor inicial propuesto: America/Mexico_City.
- Mostrar y capturar horarios locales en la zona del negocio y convertirlos correctamente a instantes.
- No codificar offsets fijos como UTC-6: las reglas históricas y futuras de zona horaria deben resolverse con la base IANA.

5. Modelo de datos conceptual

No es SQL definitivo. Es el conjunto de conceptos que el equipo debería mapear a tablas/colecciones según la tecnología y las decisiones finales.

Entidad	Propósito y datos relevantes
users	Identidad de dueño y clientes; correo único, hash de contraseña, nombre, teléfono, rol, verificación y estado.
business_profile	Perfil único del negocio; propietario, zona horaria, moneda, buffer, intervalo de slots, tiempo de retención y regla de cancelación.
services	Catálogo; nombre, descripción, precio, duración, imagen y activo/inactivo.
business_schedules	Intervalos habituales por día de semana; varias filas/intervalos por día.
schedule_exceptions	Cierres, bloqueos y horarios especiales asociados a fechas/intervalos concretos.
appointments	Cliente, servicio, snapshot de nombre/precio/duración/buffer, inicio, fin, expiración, origen y estado.
appointment_status_history	Historial de cambios de estado, actor, motivo y fecha.
payments	Importe, moneda, proveedor, identificador externo, estado y fechas relevantes.
refunds	Importe, motivo, estado, identificador de reembolso del proveedor y fechas.
payment_webhook_events	Identificador único del evento recibido, proveedor, recepción y procesamiento para idempotencia.
refresh_sessions	Sesiones/dispositivos por usuario, hash del token, expiración, última actividad, revocación y metadatos del dispositivo.
email_verification_codes	Códigos de verificación con hash/HMAC, expiración, intentos y consumo.
password_reset_codes	Códigos de recuperación con hash/HMAC, expiración, intentos y consumo.
audit_logs (recomendado)	Registro de acciones sensibles del dueño: revocaciones, cancelaciones, cambios de horario, bloqueos y reembolsos.

5.1 Datos históricos de la cita

La cita debe guardar una fotografía del servicio al momento de reservar: nombre, precio, duración y buffer. La referencia al servicio sirve para navegación/relación, pero no debe ser la única fuente histórica. Así se conserva la información correcta aunque el dueño edite o desactive el servicio después.

5.2 Relación de agenda

Para el MVP, un único calendario de trabajo es suficiente. Los horarios semanales se representan como múltiples intervalos por weekday; las excepciones representan bloqueos o jornadas especiales; las citas ocupan desde start_at hasta end_at más buffer. La disponibilidad cruza estos tres conjuntos.

6. Autenticación, autorización y sesiones

6.1 Acceso y roles

- Dos roles iniciales: OWNER y CUSTOMER.
- El registro público crea clientes; la creación/configuración inicial del propietario debe ser controlada y no permitir que cualquiera se autoasigne OWNER.
- Verificación de correo obligatoria para completar el acceso a las funciones de cliente que requieren identidad.
- Las rutas administrativas deben validar el rol en backend, no solo en la interfaz.

6.2 Tokens y sesiones

- Access token de corta duración (por ejemplo, 10–15 minutos, sujeto a decisión técnica).
- Refresh token rotativo y revocable por sesión/dispositivo.
- Persistir únicamente hash del refresh token; nunca guardar el token en texto plano.
- Al renovar, invalidar el token anterior y emitir uno nuevo.
- Al revocar una sesión, impedir tanto el uso de esa sesión como su renovación.
- Considerar revocación global por usuario como acción adicional de soporte/seguridad.

6.3 Pantalla de sesiones del administrador

Campo mostrado	Uso
Usuario	Nombre y correo del cliente.
Dispositivo / plataforma	Por ejemplo Android, iOS o navegador, cuando el cliente lo reporte.
Última actividad	Última renovación o solicitud autenticada; no equivale necesariamente a presencia en tiempo real.
Inicio y expiración	Permite reconocer sesiones antiguas o sospechosas.
Acción	Botón para revocar esa sesión específica; opcionalmente revocar todas las sesiones del usuario.

Importante: “sesión activa” significa una sesión válida/no revocada y dentro de su vigencia. No debe presentarse como prueba de que la persona está usando la app en ese segundo.

6.4 Verificación y recuperación por correo

- Códigos OTP de 6 dígitos, con vencimiento, límite de intentos, enfriamiento de reenvío y uso único.
- No almacenar OTP en texto plano; usar HMAC con secreto del servidor o un mecanismo equivalente adecuado a la baja entropía del código.
- Aplicar rate limit por IP y cuenta; respuestas genéricas para no revelar si un correo está registrado.
- La recuperación no debe iniciar sesión automáticamente sin completar el flujo de seguridad definido.

7. Superficie funcional de la API

Los nombres siguientes son agrupaciones funcionales, no rutas obligatorias. El equipo puede definir REST, RPC o GraphQL; lo importante es mantener contratos claros y autorización consistente.

Área	Operaciones que debe soportar
Auth	Registro, login, verificación de correo, reenvío OTP, refresh, logout, inicio y confirmación de recuperación.
Perfil cliente	Consultar/editar perfil y consultar historial de citas.
Negocio	Consultar perfil público y configuración necesaria para la app.
Servicios	Listar servicios activos; en admin, crear, editar, activar y desactivar.
Disponibilidad	Consultar horarios disponibles por fecha y servicio; devolver zona horaria y duración aplicadas.
Reservas	Crear retención temporal, consultar detalle, confirmar por flujo de pago, cancelar y consultar historial.
Pagos	Crear operación de pago y consultar estado; recibir webhooks firmados del proveedor.
Reembolsos	Iniciar reembolso autorizado y consultar estado.
Admin agenda	Consultar calendario, crear bloqueos, configurar horarios, gestionar excepciones y citas.
Admin usuarios	Buscar/listar clientes y listar/revocar sesiones.
Admin negocio	Editar perfil, configuración horaria, zona, buffer e intervalo de slots.

7.1 Contratos y consistencia

- Validar entradas en servidor: fechas, moneda, importes, duración, permisos y estado de las entidades.
- No aceptar del cliente móvil el precio final como autoridad; obtenerlo del servicio guardado en backend.
- Para operaciones sensibles (crear reserva, pagar, cancelar, reembolsar), considerar claves de idempotencia.
- Devolver errores de dominio identificables: horario no disponible, retención expirada, fuera de ventana de cancelación, sesión revocada, etc.
- Versionar la API o establecer una estrategia de compatibilidad para las versiones móviles instaladas.

8. Flujos críticos

8.1 Reserva y pago

Paso	Comportamiento esperado
1	Cliente selecciona un servicio y una fecha.
2	App consulta disponibilidad; backend calcula los inicios válidos.
3	Cliente selecciona un horario y solicita reservar.
4	Backend vuelve a validar horario, bloqueos y solapamientos dentro de una operación atómica.
5	Backend crea PENDING_PAYMENT con snapshot del servicio y expires_at = ahora + 10 minutos.
6	Backend crea la operación de pago con el proveedor y devuelve los datos necesarios para pagar.
7	Proveedor notifica el resultado; backend valida firma, importe, moneda, referencia e idempotencia.
8	Si el pago es válido y la retención sigue vigente, la cita pasa a CONFIRMED.
9	Si vence sin pago confirmado, pasa a EXPIRED y libera el horario.

8.2 Cancelación por cliente

- Backend verifica que la cita pertenezca al cliente y esté en un estado cancelable.
- Comprueba que falten al menos 24 horas para el inicio.
- Registra la cancelación y solicita reembolso del 100 % si existe un pago capturado.
- Registra la respuesta del proveedor y actualiza el estado del reembolso de forma asíncrona.

8.3 Cancelación por dueño

- El dueño selecciona la cita en el panel y confirma la cancelación.
- Backend registra actor OWNER, motivo y fecha; la cita pasa a CANCELLED.
- Si hay un pago completado, se crea una solicitud de reembolso por el 100 %.
- El cliente puede ver que la cita fue cancelada y el reembolso está en proceso o completado.

8.4 Bloqueo de agenda

- El dueño selecciona un intervalo para bloquear.
- Backend detecta citas que se cruzan y devuelve las afectadas antes de aplicar el bloqueo.
- El panel solicita resolver las citas afectadas (cancelar/reembolsar o reprogramar, si se habilita esa función).
- Solo después de resolver conflictos se guarda el bloqueo.

9. Procesos automáticos, integridad y operación

9.1 Tareas programadas

- Expirar reservas PENDING_PAYMENT cuyo expires_at ya pasó.
- Reintentar o conciliar operaciones de reembolso que sigan pendientes, según capacidades del proveedor.
- Detectar pagos recibidos para reservas expiradas y aplicar la política de compensación/reembolso.
- Limpiar o anonimizar códigos OTP expirados y datos temporales conforme a la política de retención.

9.2 Webhooks

- Verificar la firma del proveedor antes de procesar cualquier evento.
- Guardar el identificador único del evento y evitar procesarlo más de una vez.
- No confiar en el orden de llegada: consultar o reconciliar el estado final cuando sea necesario.
- Responder al proveedor de forma rápida y procesar tareas largas de manera asíncrona.
- No registrar secretos, datos completos de tarjetas ni información sensible en logs.

9.3 Observabilidad y auditoría

- Logs estructurados con request ID/correlation ID.
- Métricas de reservas creadas, expiradas, confirmadas, canceladas y fallos de pago/reembolso.
- Alertas para errores de webhook, reembolsos fallidos y discrepancias entre pago y cita.
- Auditar acciones administrativas sensibles: cambio de horario, bloqueo, cancelación, reembolso y revocación de sesión.
- Backups automáticos, prueba periódica de restauración y política de retención.

9.4 Seguridad básica

- HTTPS en todos los entornos expuestos.
- Hash de contraseñas con algoritmo adaptativo moderno (la elección concreta corresponde al equipo).
- Rate limiting para login, OTP, recuperación, creación de reservas y endpoints sensibles.
- Validación estricta de autorización por rol y propiedad del recurso.
- Gestión segura de secretos y credenciales del proveedor de pagos.
- Principio de mínimo privilegio para base de datos, servicios y despliegue.

10. Decisiones para revisar con el equipo

El alcance funcional está suficientemente definido para una revisión técnica inicial. Estos puntos conviene cerrarlos antes de convertir el documento en tickets y diseño físico de base de datos.

Tema	Propuesta / pregunta para el equipo
Stack	Elegir lenguaje, framework, ORM/driver, estrategia de despliegue y estructura del repositorio según experiencia del equipo.
Proveedor de pagos	Confirmar Stripe u otro proveedor, métodos habilitados en México, tiempos de liquidación, reembolsos y manejo de pagos tardíos.
Comisiones	El negocio absorbe la comisión no devuelta; confirmar las tarifas/condiciones reales de la cuenta y reflejarlo en reportes.
Reprogramación	Definir si el cliente podrá reprogramar, y bajo qué ventana. No se debe asumir hasta acordarlo.
Citas creadas por el dueño	Confirmar si el dueño puede agendar manualmente desde el panel y si esas citas requieren pago, pago en local o pueden registrarse como cortesía.
Intervalo de slots	Confirmar si el inicio de citas cada 10 minutos será fijo o configurable por el dueño.
Anticipación de reservas	Definir con cuánta anticipación se puede reservar y hasta qué horizonte futuro.
Cancelación del cliente	Se ha usado como regla 24 horas exactas antes de la cita; confirmar que esta interpretación es la deseada.
Notificaciones	Definir si V1 incluye push, correo o ambos para confirmaciones, cancelaciones y reembolsos.
Cuenta OWNER	Definir cómo se crea la primera cuenta de propietario y cómo se recupera si pierde acceso.

10.1 Recomendación de implementación por fases

Fase	Entregable
1. Fundamentos	Proyecto backend, configuración, base de datos, usuario/roles, autenticación, verificación y sesiones.
2. Catálogo y agenda	Perfil del negocio, servicios, horarios múltiples, excepciones y disponibilidad.
3. Reservas	Retenciones de 10 minutos, prevención de doble reserva, estados e historial.
4. Pagos	Integración del proveedor, webhooks idempotentes, expiración y conciliación.
5. Operación admin	Calendario, bloqueos, cancelación por dueño, reembolsos y revocación de sesiones.
6. Endurecimiento	Auditoría, observabilidad, backups, pruebas de concurrencia, seguridad y despliegue.

11. Criterios mínimos de aceptación

- Dos clientes no pueden confirmar el mismo intervalo de agenda.
- El buffer de preparación se respeta en todas las consultas y operaciones de reserva.
- Un cambio futuro en precio/duración/buffer no altera citas existentes.
- Una reserva no pagada expira a los 10 minutos y libera el horario.
- Un webhook repetido no duplica pagos, citas ni reembolsos.
- Un webhook tardío no confirma una reserva expirada de forma incorrecta.
- El cliente no puede cancelar dentro de las 24 horas previas; el dueño sí puede cancelar y el sistema solicita el reembolso completo.
- Un bloqueo que afecta citas existentes requiere resolución explícita.

- El dueño puede revocar una sesión individual desde el panel; las demás sesiones del mismo usuario permanecen vigentes.
- Las fechas se muestran correctamente en la zona horaria del negocio aunque el servidor opere en UTC.

Conclusión

La recomendación es iniciar con un backend modular, una base de datos relacional y una API compartida. La parte más delicada no es el CRUD del catálogo: es la consistencia de la agenda, la concurrencia de reservas y la conciliación entre citas, pagos y reembolsos. El equipo debería diseñar y probar esos flujos antes de conectar la experiencia final de la app.

Documento de trabajo para discusión interna. Los nombres de entidades, estados y módulos son orientativos; el equipo backend debe validar el diseño físico, contratos, proveedor de pagos y controles de seguridad antes de implementar.